ChatGPT

GeoSee Phase One Transition Diagnosis Research Report

Executive summary

This session could not directly enumerate the `GeoSee_dev` Google Drive tree through the requested connector path, nor run the active repository, so a connector-grounded code inventory, live defect reproduction log, and verified clean list could not be completed here. The only directly accessible project artifact was the uploaded Phase 1 prompt, which defines the repo path, read-only runtime hooks, invariants, and required deliverable format. As a result, the report below is a constrained but still decision-useful research synthesis: it extracts the strongest evidence available from the prompt, converts that into a concrete Phase 1 diagnosis framework, and layers in current best practices from primary tooling documentation and closely related graphics-stack projects. filecite turn0file0 L35-L68 filecite turn0file0 L71-L92

The strongest provisional conclusion is **harden incrementally, not rewrite yet**. That recommendation is necessarily provisional, but it is grounded in the architecture clues embedded in the prompt: the existing runtime already appears to have dedicated modules for transition styles, layer tweening, effective-layer resolution, entity policy, render-graph adaptation, execution planning, opacity execution, commit gating, frame signatures, and product-family registration. That is not the shape of a codebase that obviously lacks a transition architecture; it is the shape of a codebase that more likely needs a smaller number of high-leverage fixes plus a durable per-family regression-test net. The Phase 1 prompt itself also explicitly frames this pass as evidence gathering before any redesign decision. filecite turn0file0 L11-L22 filecite turn0file0 L26-L31 filecite turn0file0 L55-L67

The most important practical deliverable, given the access limits, is therefore a **ready-to-run Phase 1 work package**: a page-agnostic defect-family matrix, a per-family regression-test net, a provisional harden-vs-rewrite comparison, and a grounded outline for a single Phase 2 design that uses the existing scaffolding rather than discarding it. That matches the prompt's emphasis on converging diagnosis, page-agnostic defect families, read-only evidence collection, and the regression-test net as the durable mechanism that keeps future entities from re-breaking transitions. filecite turn0file0 L91-L92 filecite turn0file0 L112-L149 filecite turn0file0 L153-L178

A downloadable Markdown copy of this report is available here:
GeoSee_Phase_1_Diagnosis_Research_GPT52T_2026-06-14.md

Evidence base and operational limits

The uploaded prompt establishes the working context clearly. Phase 1 is a **read-only** pass that must answer two questions with evidence: what transition behavior is still broken now after DEC-164 and DEC-165, and whether the smallest sufficient path is incremental hardening or re-architecture. It also states that Phase 2

design should not begin until Phase 1 determines whether redesign is warranted.
filecite turn0file0 L11-L22

The same prompt identifies the expected active repository as `C:\GeoSee\GeoSee_Dev` on branch `main`, says the dev server should serve that tree, and points to read-only runtime hooks through `window.__pageController` and `window.__GEOVOX_DIAGNOSTICS__.getSceneHandle().manager`. It also enumerates the approved checks and tools for evidence gathering, including `npm run dev`, `npx tsc --noEmit`, `node tools/validate-project-pages.mjs`, `npm run check:workspace`, and the deck-state tools for capture and analysis. filecite turn0file0 L35-L51

The prompt also provides the strongest direct evidence about the current transition runtime shape by naming the files that must be read first. Those names indicate a system with a central `SceneManager`, a transitions subsystem, section and overlay controllers, a UI legend component, an execution-plan stage, a commit gate, a frame-signature mechanism, and a family registry. From filenames alone, that strongly suggests an already-decomposed transition runtime rather than an ad hoc monolith. This is an inference from the file list, not a verified code inventory. filecite turn0file0 L55-L68

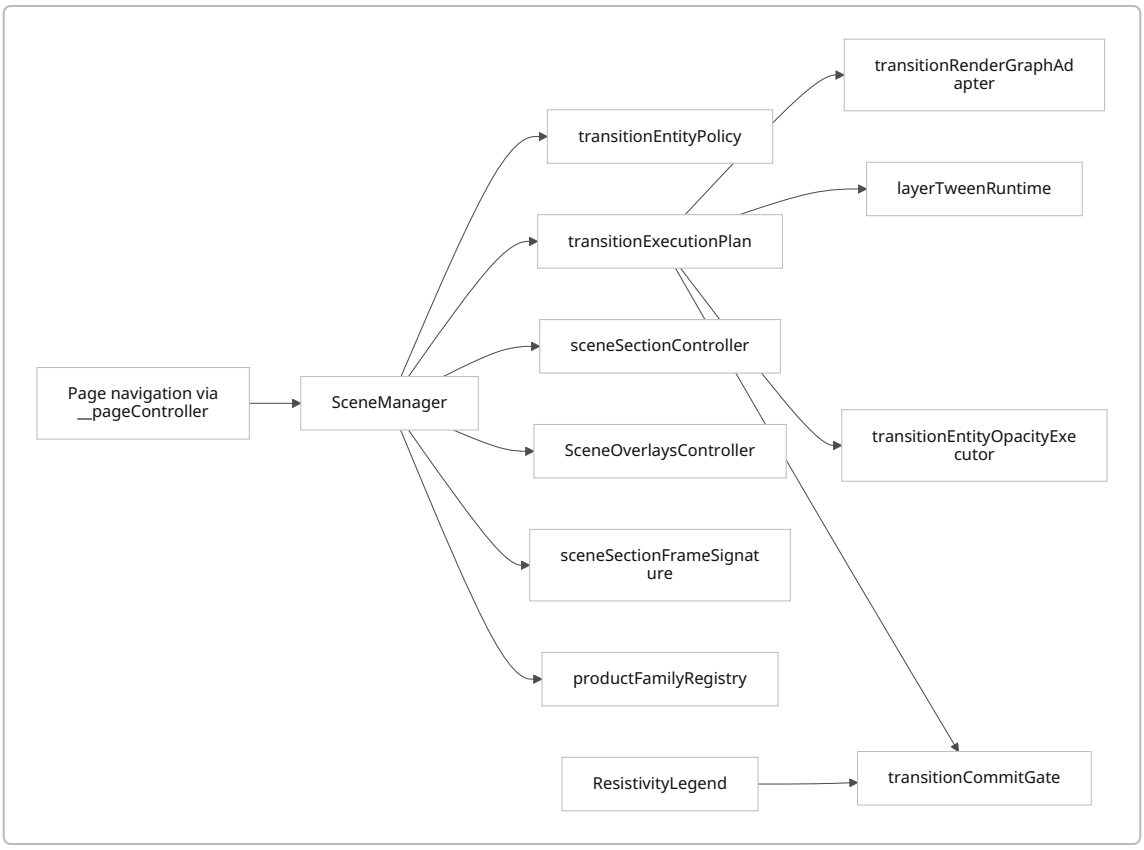
A second hard evidence block is the invariant list. The uploaded prompt explicitly marks the MT Resistivity Cells slider as a known-good reference, calls the MT full-density scalar cloud a protected success path, requires surviving sections to move rather than dissolve, requires section-helper handedness to be right-handed, and treats helper stability, ISO opacity composition, discrete color/material channels, single commit ownership, and overlay/camera synchronization as correctness rules. The prompt references an additional spec file for full details, but that file was not available in this session, so only the invariants reproduced in the uploaded prompt were treated as binding. filecite turn0file0 L71-L92

The largest limitation is that **none of the repo-grounded tasks could be completed live here**: no Google Drive inventory via connector, no repo file scrape, no runtime reproduction, no screenshots from the live app, and no validation of whether the already-landed fixes still hold. Because of that, every family-status judgment below is marked as either **prompt-specified**, **inferred**, or **needs runtime evidence**, never as independently reproduced. That status language is itself consistent with the prompt's requirement to distinguish "reproduced," "not reproduced," and "needs runtime evidence." filecite turn0file0 L96-L108 filecite turn0file0 L153-L165

Inferred application architecture

The accessible evidence points to a web-based TypeScript application with a 3D scene runtime and TSX-based UI surfaces. That is an inference from the presence of `.ts` and `.tsx` source paths, Node-driven dev and check commands, and browser-exposed diagnostic objects. It is not yet a verified dependency inventory. filecite turn0file0 L35-L51 filecite turn0file0 L55-L68

The currently inferable architecture looks like this:



That inferred decomposition matters because it changes the default recommendation. A runtime that already contains an execution-plan stage, commit gating, family registration, and frame signatures usually benefits more from tightening ownership boundaries, fixing identity/landing semantics, and backstopping behavior with family-specific fixtures than from a greenfield rewrite. The prompt's own inventory categories point in the same direction: section move/dissolve, scalar layers, seismic entities, overlays, legends, camera, commit/settle, readiness/attach, and interruption/replan all read like **behavior families passing through a shared framework**, not like unrelated one-off effects. 🗄filecite🗄turn0file0🗄L60-L67🗄 🗄filecite🗄turn0file0🗄L116-L130🗄

A useful analogue comes from deck.gl. Its official transition model explicitly separates camera transitions from layer-property transitions, defines interruption modes such as break, snap-to-end, and ignore, and documents a "set-and-forget" model in which transition parameters are frozen at start and late mutations are treated as interruptions. That is relevant because the GeoSee prompt explicitly includes interruption/replan, camera synchronization, and commit/settle as transition families. The architectural lesson is not to copy deck.gl wholesale, but to make interruption semantics and start-of-transition ownership explicit in the existing execution-plan/commit-gate scaffolding. [1]

Phase One diagnosis framework and provisional inventory

The prompt requires Phase 1 to be page-agnostic, family-based, evidence-driven, and explicit about what is broken versus what is clean. It also requires verification of already-landed fixes without relitigating them as novel issues. Because live reproduction was unavailable here, the most useful substitute is a **provisional**

family matrix that can be executed unchanged once the repo becomes accessible. filecite turn0file0 L96-L108 filecite turn0file0 L112-L149

The table below should be read as a **diagnosis queue**, not as a confirmed defect list.

Behavior family	Prompt-backed invariant or reference	Primary inferred owners	Current session status	Provisio harden v rewrite c
Section move and dissolve	Surviving frames must move; only added/ removed/identity-changed frames dissolve; DEC-164 must still hold. filecite turn0file0 L83-L85 filecite turn0file0 L100-L102	`sceneSectionFrameSignature.ts`, `sceneSectionController.ts`, `transitionExecutionPlan.ts`, `layerTweenRuntime.ts` filecite turn0file0 L60-L67	Needs runtime evidence	Harden
Section helper orientation and basis stability	Helper basis must be right-handed; Y helper world bbox flat in Y; determinant +1; no per-tick rebuild. filecite turn0file0 L85-L90 filecite turn0file0 L103-L104	`sceneSectionController.ts`, helper/ runtime code near section frames filecite turn0file0 L66-L67	Needs runtime evidence	Harden
MT Resistivity Cells slider	Known-good local scalar-style interpolation, ~1.5s, persists only `scalarLegendVisible` and `scalarInclusionMode`; should not be “fixed.” filecite turn0file0 L77-L80	`ResistivityLegend.tsx`, scalar-family transition code filecite turn0file0 L66-L67	Prompt-specified positive control, not independently reproduced	Preserve use as referenc
MT full-density scalar cloud	Protected success path; isolines enter on page clock; no commit-time replay pop. filecite turn0file0 L81-L82	`transitionExecutionPlan.ts`, `transitionCommitGate.ts`, family registry filecite turn0file0 L63-L65	Prompt-specified positive control, not independently reproduced	Harden around i not repla

Behavior family	Prompt-backed invariant or reference	Primary inferred owners	Current session status	Provisio harden rewrite
MT volume, threshold, and slices	No direct status in prompt; grouped as an inventory family. filecite turn0file0 L117-L120	`transitionEntityPolicy.ts`, `transitionEffectiveLayersForPage.ts`, `transitionRenderGraphAdapter.ts` filecite turn0file0 L61-L64	Needs runtime evidence	Harden identity attach semanti prove fundam inconsis
Seismic volume, sections, and figures	Execution-plan seismic entities previously called out as a base invariant. filecite turn0file0 L86-L89	`transitionExecutionPlan.ts`, `transitionRenderGraphAdapter.ts`, family registry filecite turn0file0 L63-L65	Needs runtime evidence	Harden
Section-ISO overlays	ISO opacity must compose rather than overwrite; overlays must run on transition clock. filecite turn0file0 L86-L90	`transitionEntityOpacityExecutor.ts`, `SceneOverlaysController.ts`, commit gate filecite turn0file0 L64-L67	Needs runtime evidence	Harden
Point clouds	Listed as behavior family only. filecite turn0file0 L117-L120	Family registry, render-graph adapter, effective layers filecite turn0file0 L61-L65	Needs runtime evidence	Harden
Boreholes	Listed as behavior family only. filecite turn0file0 L117-L120	Family registry, render-graph adapter, attach/ readiness logic filecite turn0file0 L61-L65	Needs runtime evidence	Harden
Legends and labels	Legend writes must not persist onto nondisplayed page during transition. filecite turn0file0 L107-L108	`ResistivityLegend.tsx`, `transitionCommitGate.ts` filecite turn0file0 L64-L67	Needs runtime evidence	Harden
Camera and overlays	Overlays and camera must remain on transition clock; orbit radius clamp must still hold. filecite turn0file0 L89-L90 filecite turn0file0 L105-L106	`SceneManager.ts`, overlays controller, camera runtime around manager hook filecite turn0file0 L42-L44 filecite turn0file0 L66-L67	Needs runtime evidence	Harden

Behavior family	Prompt-backed invariant or reference	Primary inferred owners	Current session status	Provisio harden v rewrite c
Fog and background	Listed family only. filecite turn0file0 L119-L120	Scene manager / render graph	Needs runtime evidence	Harden
Commit and settle	Single commit owner and deferred flush suppression; no commit-time replay pop. filecite turn0file0 L86-L90	`transitionCommitGate.ts`, `transitionExecutionPlan.ts` filecite turn0file0 L63-L65	Needs runtime evidence	Harden
Readiness and attach	Listed family only. filecite turn0file0 L120-L120	Render-graph adapter, family registry, attach gating	Needs runtime evidence	Harden escalate if attach semanti differ irreconc by family
Interruption and replan	Listed family only; analogous to explicit interruption modes in mature transition stacks. filecite turn0file0 L120-L130 [1]	`transitionExecutionPlan.ts`, `transitionCommitGate.ts`, page controller hooks filecite turn0file0 L41-L44 filecite turn0file0 L63-L65	Needs runtime evidence	Harden replan c be mode as plan invalidat plus rest

Two rows deserve special treatment. The MT Resistivity Cells slider and MT full-density scalar cloud are not merely families to test; the prompt explicitly identifies them as **positive references**. Those should become fixture baselines and red-flag canaries in CI, not generic cases mixed into a random transition pool. filecite turn0file0 L77-L82

Because no live reproduction was possible here, a true clean list cannot be claimed. The closest substitute is a **spec positive-control list**: the two MT references above, plus the already-landed verification set that Phase 1 is supposed to confirm rather than re-argue—DEC-164, DEC-165, orbit-radius clamp, and the legend write landing gate. These items are **not independently verified in this session**. They remain the first candidates for live confirmation once repo access is available. filecite turn0file0 L96-L108

The highest-value method notes from the prompt can be operationalized immediately. The runtime diagnosis loop should start with `npm run dev`, use `window.__pageController` to perform exact navigation, inspect `window.__GEOVOX_DIAGNOSTICS__.getSceneHandle().manager` during enter/landed/settled phases, and then collect either matrix captures, screenshots, or numeric readbacks such as bbox extent, determinant, opacity, clip state, and commit ownership. That procedure is directly aligned with the prompt's evidence requirements. filecite turn0file0 L40-L51 filecite turn0file0 L121-L130

The regression-test net should mirror the family structure. Graphics-heavy systems benefit from splitting tests into three layers: fast logic tests for plan construction and identity rules; browser-level fixture tests for DOM/UI behavior and diagnostic state; and golden-image or screenshot tests for scene results. Playwright's official guidance emphasizes testing user-visible behavior, isolation, auto-retrying assertions, visual comparisons, and trace capture, while deck.gl's official testing guidance explicitly separates prop-update unit tests from browser-driven golden-image integration tests because GPU behavior and canvas rendering are difficult to validate purely in Node. [2]

A practical per-family regression-test net is below.

Family	Synthetic fixture focus	Core assertions	Recommended tool	Evidence artifact
Section move/ dissolve	Same-axis surviving plane + toggled orthogonal plane	Survivor world position interpolates; added/ removed plane dissolves; no snap	Vitest for execution-plan math; Playwright for live transition	Numeric bbox/ determinant readback plus screenshot diff
Helper orientation	Canonical X/Y/Z section helpers	Determinant remains +1; Y helper bbox flat in Y; no per-tick object rebuild	Vitest plus Playwright inspector hook	Numeric assertions and trace
Scalar positive controls	Resistivity slider and full-density scalar cloud fixtures	Duration and persistence contracts; no replay pop; isolines enter on page clock	Playwright or Vitest Browser Mode	Trace, screenshot baseline, logged landed state
Legends and labels	Transition into and away from legend-bearing page	Writes do not land on nonvisible page; `legendWritesLanded` semantics preserved	Playwright	DOM assertions plus trace
Camera and overlays	Transition with simultaneous overlay and camera movement	Overlay state, camera state, and scene transition clock remain synchronized	Playwright	Trace plus sampled manager state
Commit/ settle	Fast successive transitions and mid-flight interruption	Single commit owner; no double-land; no deferred-flush leak	Vitest for plan/ commit state and Playwright for runtime	Trace plus timeline log
Readiness/ attach	Late-attaching family entity	No premature visibility; no pop-in at commit	Playwright	Trace, screenshot, readiness log

Family	Synthetic fixture focus	Core assertions	Recommended tool	Evidence artifact
Interruption/ replan	Rapid `next()/ prev()/ goto()` while in flight	Explicit BREAK vs SNAP style behavior; no blended corrupted end state	Playwright	Trace plus state snapshots

That net should be run under a fixed browser/OS baseline for visual checks. Playwright's documentation notes that screenshots differ across browsers and platforms and that per-project snapshot paths should be used accordingly; its CI guidance also recommends a stable environment, sequential workers in CI for reproducibility, and report artifact uploads. [3]

Harden vs rewrite recommendation

The prompt asks for one recommendation, not an indecisive blend. Under the currently available evidence, the right recommendation is **incremental hardening with a strong regression-test net**. That recommendation is supported by three things at once: the prompt's own "diagnosis first" framing, the breadth of existing transition scaffolding suggested by the file map, and the lack of direct evidence that the current model is structurally incapable of expressing the listed invariants. filecite turn0file0 L26-L31 filecite turn0file0 L55-L67 filecite turn0file0 L137-L149

A rewrite should be considered only if live Phase 1 evidence shows one or more of the following: first, core invariants fail across several families for the same root cause in identity, landing, or interruption semantics; second, the current execution-plan/commit-gate model cannot express family policies without page-pair special casing; third, attach/readiness and interrupt/replan produce nondeterministic landed states even after targeted fixes; or fourth, the regression net itself cannot be written family-first without reproducing the same logic branches as the production runtime. Those are the signs that the issue is model-level rather than implementation-level. This threshold is an inference from the prompt's deliverable goals and from how mature transition stacks formalize interruption, ownership, and family policy. filecite turn0file0 L141-L149 [4]

The option comparison looks like this:

Option	Why it fits or does not fit current evidence	Cost shape	Risk shape	When to choose
Harden incrementally	Existing subsystem already appears decomposed into plan, policy, commit, opacity, family registry, and frame-signature pieces	Lower immediate cost; test net investment pays back quickly	Lower delivery risk; medium risk of hidden coupling	Default choice now

Option	Why it fits or does not fit current evidence	Cost shape	Risk shape	When to choose
Rewrite around a unified model	Only justified if live inventory proves cross-family failures stem from one irreducible model flaw	Highest near-term cost; long freeze risk	Highest regression risk during migration	Only after evidence crosses the thresholds above

The incremental hardening path should be ordered by leverage. The recommended order is: **section identity and motion**, **commit/settle ownership**, **camera/overlay clocking**, **legend landing gates**, **interruption/replan semantics**, then **family-specific attach/readiness edge cases**. That order follows directly from the base invariants and from the fact that the corresponding modules look centrally shared rather than family-local. filecite turn0file0 L83-L90 filecite turn0file0 L100-L108 filecite turn0file0 L60-L67

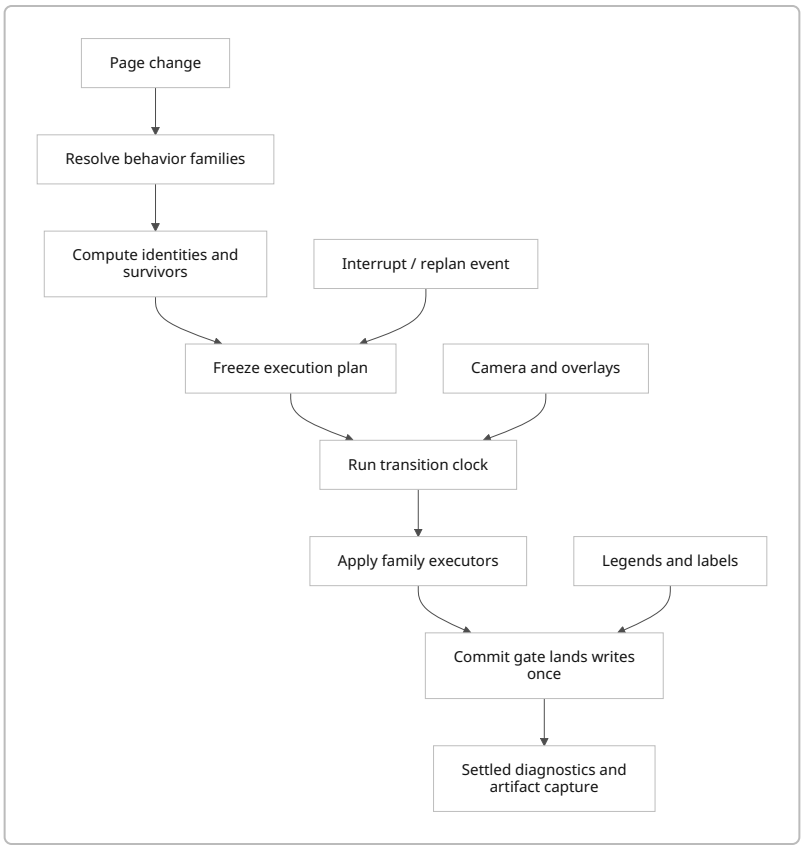
Phase Two grounded design and red-team points

The uploaded prompt properly says Phase 2 design should wait until Phase 1 decides whether redesign is warranted. Even so, the user also asked for a grounded Phase 2 concept. The least risky Phase 2 concept is therefore **not a rival engine**. It is a unification layer built on the existing scaffolding already named in the prompt. filecite turn0file0 L3-L7 filecite turn0file0 L21-L22

The grounded idea is a single transition contract with five explicit stages:

1. **Family resolution** through `productFamilyRegistry.ts` and `transitionEntityPolicy.ts`.
2. **Identity and survivorship** through `sceneSectionFrameSignature.ts` and equivalent signatures for other families.
3. **Execution-plan freeze** through `transitionExecutionPlan.ts`, producing one immutable plan for the active transition.
4. **Clocked execution** through `layerTweenRuntime.ts`, `transitionEntityOpacityExecutor.ts`, and overlay/camera synchronization.
5. **Landing and settle ownership** through `transitionCommitGate.ts`, including all landed-write gates such as legends and labels. filecite turn0file0 L60-L67

A Phase 2 architecture built this way would look like this:



That design is consistent with the testable patterns documented by similar projects. deck.gl separates family-like layer transitions from camera transitions, freezes transition properties at the start of a run, and defines explicit interruption handling. Playwright and Vitest both support traceable, browser-grounded test execution for visually meaningful flows, which is critical if GeoSee uses this structure to encode page-family transitions in deterministic fixtures. [5]

The red-team critique points should focus on the places most likely to hide model flaws:

Red-team question	Why it matters
Does identity remain stable across families, or only for sections?	If identity differs by family, “move vs dissolve” may remain inconsistent.
Is the execution plan truly immutable after transition start?	If late mutations leak in, interruption bugs will masquerade as family bugs.
Can landed writes be proven single-owner for UI and scene state together?	This is the hinge for legends, overlays, and commit-time replay pops.
Are camera and overlays clocked by the same state machine as scene layers?	If not, “smooth” transitions can still desynchronize visibly.
Can attach/readiness be represented as family policy instead of page-pair exceptions?	If not, the runtime is likely drifting back toward deck-specific brittle logic.

Red-team question	Why it matters
Is interruption semantics explicit for BREAK, SNAP, and IGNORE-like cases?	If not, rapid navigation bugs will persist or reappear under load.

Tooling, CI, and testing best practices

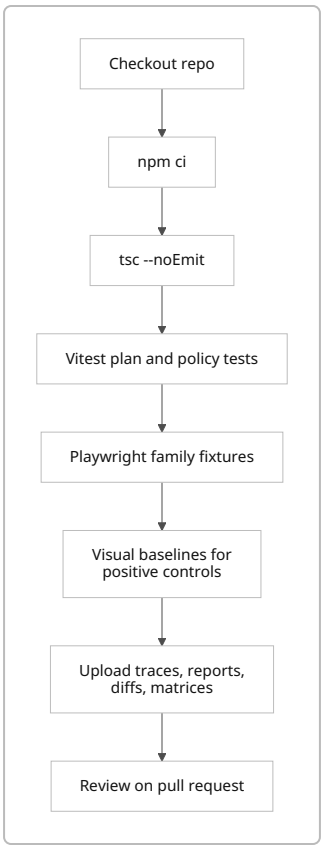
The most actionable best-practice stack for a TypeScript-heavy, browser-rendered, visually sensitive app like the one implied by the prompt is a **layered testing toolchain** rather than a single framework. The quickest path is to keep `npx tsc --noEmit` as the static type gate, use Vitest for fast plan/policy/unit assertions, use Playwright for browser-level transition fixtures, and reserve visual regression for the important positive controls and highest-risk families. TypeScript's `noEmit` mode is designed exactly for type-check-only validation; Vitest is designed to reuse Vite config and supports TypeScript, browser mode, snapshots, coverage, and isolated parallel execution; and Playwright supplies auto-retrying user-facing assertions, visual comparisons, and traces for debugging CI failures. filecite turn0file0 L49-L51 [6]

For UI-facing assertions, Testing Library's guidance still applies even in a scene-heavy application: tests should resemble how the software is actually used, and component tests should avoid implementation-detail coupling. That principle is especially important for legend state, labels, navigation events, and page-level controls, where brittle DOM or function-name assertions are more likely to rot than user-visible checks. [7]

For graphics-heavy layers, the best precedent is deck.gl. Its official testing guide says GPU/browser behavior is harder than regular JavaScript to test, explains why canvas output cannot be perfectly verified in Node alone, and uses both prop-update tests and browser-based golden-image tests. That matches GeoSee's needs closely enough to adopt the same layered pattern even if the rendering stack is different. [8]

For CI, the most reliable near-term setup is a GitHub Actions pipeline that installs dependencies with `npm ci`, runs type checks and fast tests first, then runs Playwright tests in a stable environment with artifacts preserved. GitHub's Node.js workflow guidance supports matrix builds across Node versions, while Playwright's CI guidance recommends sequential workers in CI for reproducibility and includes artifact upload patterns for `playwright-report`. GitHub's artifact system also supports retention settings and digest validation, which is useful for preserving transition traces and screenshot baselines as durable review evidence. [9]

A practical CI flow should look like this:



Three implementation details are worth enforcing from day one:

Practice	Why it matters	Source basis
Keep browser/OS baselines fixed for screenshot tests	Visual snapshots differ by platform and browser	Playwright notes platform/browser variance for screenshots. [10]
Record traces on transition-family failures	Traces expose actions, source lines, logs, network, attachments, and image diffs	Playwright Trace Viewer is designed for CI failure analysis. [11]
Prefer low-mock, behavior-visible tests	These give more confidence and survive refactors better	Playwright and Testing Library both emphasize user-visible behavior and avoiding implementation-detail coupling. [12]

Two analogous project practices are also worth copying. First, deck.gl's explicit interruption semantics are a strong model for the GeoSee interruption/replan family. Second, Mermaid's own documentation highlights visual regression as part of its review and release flow, which is a useful reminder that diagrammed architecture and visual baselines can coexist in a healthy CI process. [13]

Downloadable markdown

The requested Windows destination path could not be written directly from this session. A download-ready Markdown version was created in the sandbox instead and can be moved locally into `C:\GeoSee\GeoSee_Dev\Transition Architecture\Phase_1_Diagnosis\` after download.

Download: GeoSee_Phase_1_Diagnosis_Research_GPT52T_2026-06-14.md

Suggested target filename once moved locally:
`GeoSee_Phase_1_Diagnosis_GPT52T_2026-06-14.md`

Internal Report ID: `GPT52T-PHASE1-GEOSEE-2026-06-14-A`

---

1 4 5 13 https://deck.gl/docs/developer-guide/animations-and-transitions
https://deck.gl/docs/developer-guide/animations-and-transitions

2 12 https://playwright.dev/docs/best-practices
https://playwright.dev/docs/best-practices

3 10 https://playwright.dev/docs/test-snapshots
https://playwright.dev/docs/test-snapshots

6 https://www.typescriptlang.org/tsconfig/noEmit.html
https://www.typescriptlang.org/tsconfig/noEmit.html

7 https://testing-library.com/docs/guiding-principles
https://testing-library.com/docs/guiding-principles

8 https://deck.gl/docs/developer-guide/testing
https://deck.gl/docs/developer-guide/testing

9 https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs
https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs

11 https://playwright.dev/docs/trace-viewer
https://playwright.dev/docs/trace-viewer